实验工作 1
安全 REST API、CI/CD 与真实 CVE 分析

FastAPI + JWT · GitHub Actions · CVE-2021-41773 复现与修复

姓　名	Zhang San
学　号 (IDU)	12345678
Variant	1
仓库	github.com/huangweijie429/Information-Security-Lab1

1. 工作目的

本工作覆盖一个 API 的完整安全生命周期：按 OWASP Top 10 设计防护、把安全工具接入 CI/CD、 在受控环境（Vulhub 同款配置）中复现一个真实 CVE、分析根因、实施修复并验证攻击失效与功能回归。

2. API 设计与实现

技术栈：**FastAPI + SQLite（标准库 sqlite3）+ bcrypt + PyJWT**，代码位于 `work1-api/`。

方法	路径	鉴权	说明
GET	`/health`	否	存活探针
POST	`/auth/login`	否	接受 `{username, password}`，返回 JWT
GET	`/api/data`	Bearer	帖子列表，支持 `?q=` 过滤
GET	`/api/me`	Bearer	当前用户资料（自选的第三个接口）

3. 安全措施

风险	实现方式
SQL 注入	所有语句使用 `?` 参数化查询（登录查询、`q` 过滤），不做字符串拼接； 用户名额外做白名单校验 `^[A-Za-z0-9_.-]+$`
XSS	返回前用 `html.escape()` 转义所有用户内容；回归测试断言 `<b>` 变成 `<b>`
认证失败	bcrypt（cost 12）存口令哈希；HS256 JWT 带 `exp` / `iss`； 所有受保护端点经 `HTTPBearer` 依赖校验，缺失/伪造 token 返回 401

4. CI/CD 流水线

`.github/workflows/ci.yml` 在每次 push 与 pull request 触发，依次执行：

1. **SAST**：`bandit -r app`（0 告警）
2. **SCA**：`pip-audit -r requirements.txt --strict`（No known vulnerabilities）
3. **测试**：`pytest -q`（10 个用例全过，覆盖 SQLi/XSS/鉴权回归）

运行记录：仓库 Actions 页，例如 run 39257053599（首次全绿）。

5. CVE-2021-41773 复现

目标：Apache HTTP Server 2.4.49 路径穿越（同时可导致 CGI 命令执行）。 环境按 Vulhub 同款配置构建（`work1-cve/Dockerfile`）：启用 `mod_cgi` 与 `ScriptAlias`，并把 `<Directory />` 设为 `Require all`

`granted` 。

请求	漏洞版 2.4.49	修复版 2.4.51
`GET /cgi-bin/.%2e/.%2e/.%2e/.%2e/etc/passwd`	**HTTP 200**，返回 `root:x:0:0:root:/root:/bin/bash`	HTTP 400，无文件内容
`POST …/bin/sh`，body 为 `echo Content-Type: text/plain; echo; id`	**HTTP 200**，`uid=1(daemon) gid=1(daemon)`	HTTP 400，无输出
`GET /cgi-bin/hello.sh`（回归）	200 hello from cgi	200 hello from cgi（功能未破坏）

完整命令与输出见 `work1-cve/evidence/01-05-*.txt` 。

6. 根因分析

2.4.49 重写了路径规范化（`ap_normalize_path`）：它只对「从文档根开始」的路径展开 `%2e` 段， 没有拒绝逃逸出文档根的 `..` 序列，于是 `/.%2e/.%2e/…` 在鉴权检查之后、文件查找之前被规范化成 `../../..`，绕过了访问控制。两个配置选择把「文件读取」放大成「命令执行」：

1. `<Directory />` 允许访问整个文件系统，使逃逸路径通过鉴权；
2. 启用了 `mod_cgi` 与 `ScriptAlias`，被穿越到的可执行文件会以请求体作为输入被执行。

因此这是一个**输入处理缺陷**（用户可控路径未完整规范化），而非内存安全问题。

7. 修复与验证

层次	改动	为什么有效
补丁	镜像升级 `httpd:2.4.49 → 2.4.51`	2.4.51 拒绝任何规范化后逃逸文档根的路径，并封堵 2.4.50 的双重编码 `%252e` 绕过
加固	`<Directory />` 改为 `Require all denied`，CGI 仅在 cgi-bin 内	移除第二个前提：即使将来再出现规范化漏洞，穿越路径也无法通过鉴权

修复后同一 PoC 返回 400，正常 CGI 与首页仍可用（回归通过）。 `poc.sh` 可一条命令对任意实例做验证： 漏洞版退出码 1，修复版 0。

8. 结论

「参数化查询 + 输出转义 + 强认证」能把应用层的注入与脚本执行风险压到很低，但它无法阻止 **基础设施组件自身的缺陷**。这次 CVE 的教训是双层的：及时打补丁（消除已知漏洞）， 以及最小权限配置（即使补丁缺失，也把影响面从 RCE 压到 403）。两者叠加，才让回归测试同时证明 「攻击失效」和「功能未坏」。